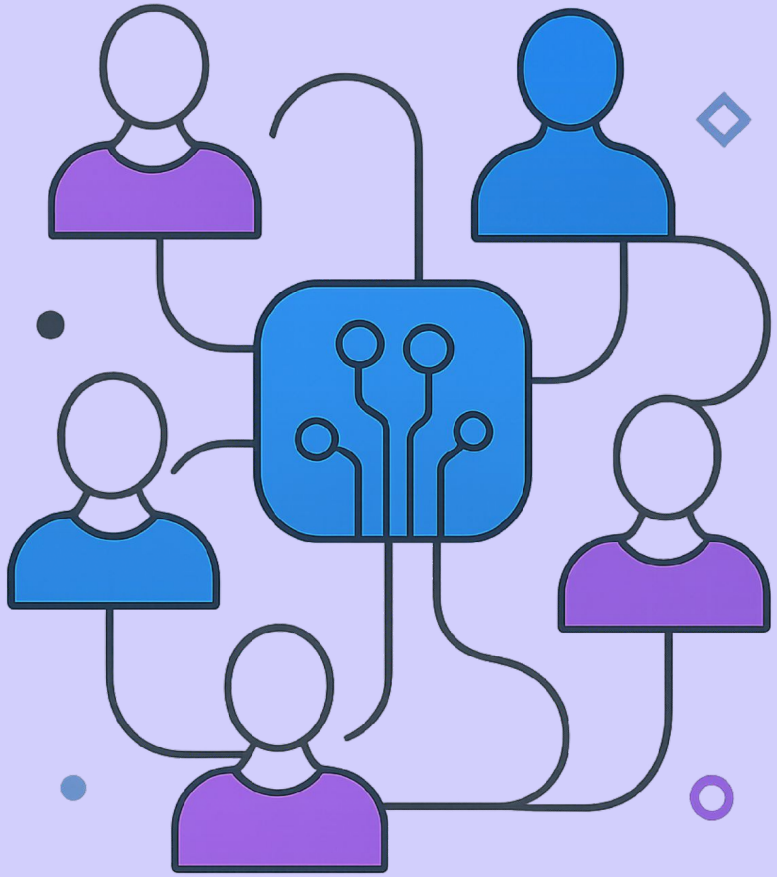
Portable AI Agents : Agent Server + Agent-Native Browser + NandaIndex for Discovery

OpenClaw can evolve from a skill-augmented local agent into an **agent-native browser for the open Agentic Web** — downloading, verifying, and running signed agent packages the same way a browser renders webpages.



Overview: Portable AI Agents

Agent Browser + Agent Server + NANDA Index (DNS+CA)

OpenClaw Browser

Package download + verification, permission UI, LLM router binding, local tool execution, memory vault, budget enforcement, audit log, plugin system

Agent Origin Server

Serves agent.package + AgentFacts, remote tools + private APIs, eval results, attestations, package updates, billing, and publisher-side policy

NANDA Index

Agent discovery + identity, package location, publisher verification, registry federation, trust metadata, version pointers, capability metadata

CORE IDEA

The Web Analogy

Today's browser downloads HTML, renders it locally, and calls back to servers for dynamic data. The same architectural move can happen for AI agents.

Instead of visiting

ramesh.com/index.html

An agent-native browser visits:
ramesh.com/agent.package
or resolves via NANDA:
travel@ramesh.com

The Simplest Transformation

OpenClaw → agent-native browser
skill.md → signed skill
signed skills + manifest + tools + policy + evals → **agent.package**
agent.package + NANDA + OpenClaw runtime → **open Agentic Web**

SECTION 1

Classical Web vs. Agentic Web

HTML → agent.package
Portable, signed, executable agent bundles

URL → NANDA Locator
Agent addresses like travel@ramesh.com

DNS → NANDA Index
Discovery and resolution layer for agents

Cookies → Memory Capsules
Scoped, user-controlled agent memory

Browser → OpenClaw
User-sovereign agent runtime

PageRank → AgentRank (Trust based)
Reputational ranking, Capability, eval, and trust graph

The browser becomes a **user-sovereign agent runtime**.

Classical Web	Agentic Web
HTML	agent.package
JavaScript	signed tools, skills, and runtime modules
Browser	OpenClaw agent-native browser
URL	NANDA locator / agent URL
DNS	NANDA Index plus DNS fallback
HTTP server	agent origin server
Cookies	scoped agent memory capsules
Browser permissions	tool, memory, model, data, and budget permissions
Browser plugins	OpenClaw plugins and tool adapters
Search engine	agent search / agent finder
PageRank	AgentRank / capability graph / trust graph
Web app	downloadable, executable agent network

SECTION 2

OpenClaw as the Agent-Native Browser

OpenClaw should become the **"Chrome of agents"** — not merely running one local agent, but the place where users discover, download, verify, run, compose, and audit third-party agents.

Verification & Security

Signature verification, permission review, sandbox enforcement, and audit logs

Identity & Authority

API key protection, personal data access, budget enforcement, and user consent

Agent Orchestration

LLM/router binding, tool execution, local vs. remote routing, agent-to-agent communication

The agent may be downloaded from the internet, but the **authority to act comes from the user's OpenClaw browser**.

SECTION 3

From skill.md to Signed Skill

Why skill.md Is Insufficient

A raw skill.md is too weak for internet-scale execution. It does not reliably define authorship, signatures, permissions, tool bindings, memory access, eval results, versioning, or dependencies.

A Signed Skill Contains

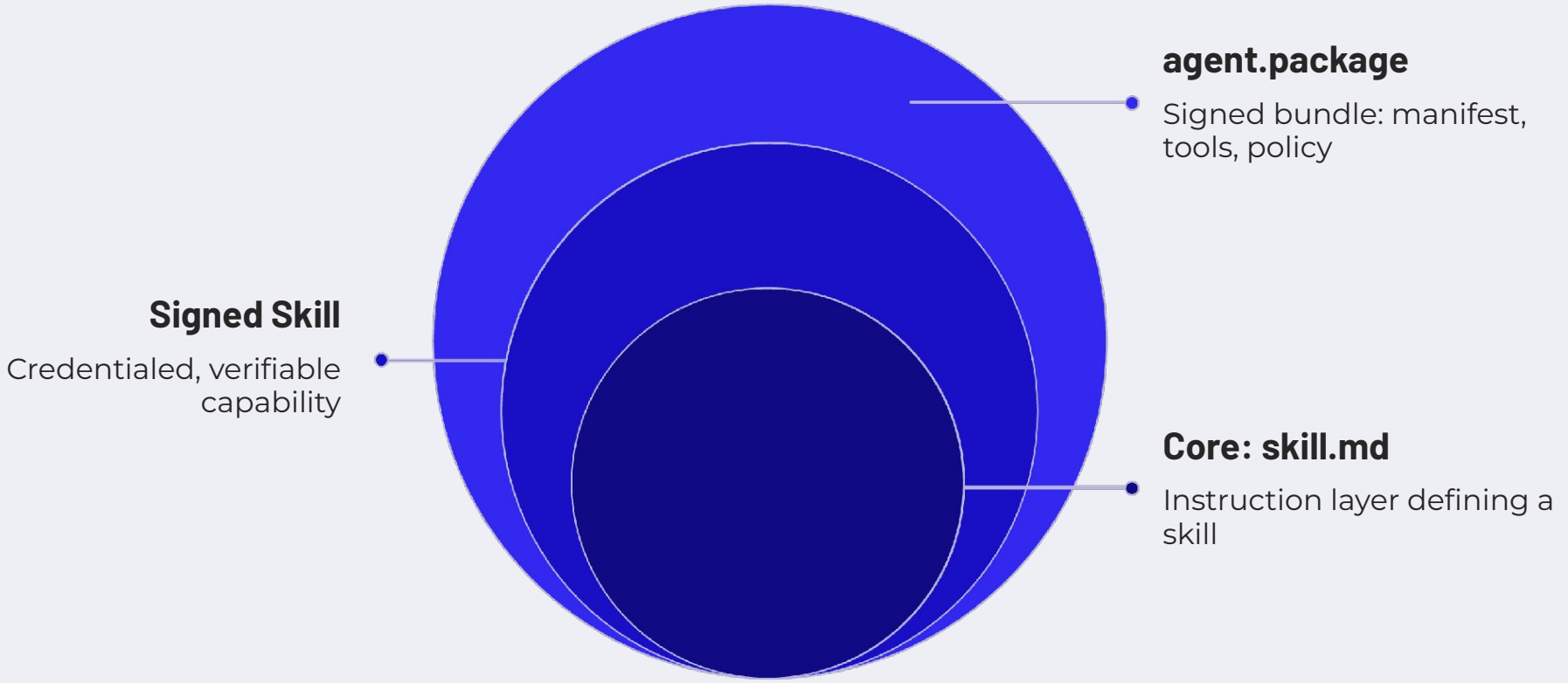
- skill.md + manifest
- Author identity + signature
- Version + permissions
- Tool bindings + dependencies
- Evals + attestations
- Allowed runtime capabilities

The markdown remains valuable — humans can read and edit it. But it should become the **instruction layer inside a signed package**, not the execution unit itself.

SECTION 4

From skill.md to agent.package

Like a Java JAR file, an agent.package is a disciplined signed bundle — not just a HOCON or markdown file.



A travel.agent.package includes skills/, tools/, policy/, evals/, ui/, signatures/, and attestations/ — everything needed for safe, verifiable execution.

SECTION 4 DETAIL

agent.package Structure

MANIFEST.agent + agent.hocon

Declares the agent graph, identity, and configuration

skills/ + tools/

Signed skill.md files and tool definitions (calendar, airline-search, payment)

policy/

permissions.json, network-policy, memory-policy, budget-policy

evals/ + ui/

Safety evals, booking evals, approval forms, and trip summary widgets

signatures/ + attestations/

Publisher sig, skill-attestor sig, SBOM, provenance, and safety review

```
travel.agent.package
│
├── MANIFEST.agent
├── agent.hocon
├── agent.json
├── skills/
│   ├── planning.skill.md
│   ├── booking.skill.md
│   └── refund.skill.md
├── tools/
│   ├── calendar.tool.json
│   ├── airline-search.tool.json
│   └── payment.tool.json
├── policy/
│   ├── permissions.json
│   ├── network-policy.json
│   ├── memory-policy.json
│   └── budget-policy.json
├── evals/
│   ├── basic-trip-planning.eval.json
│   ├── booking-safety.eval.json
│   └── refund-boundary.eval.json
├── ui/
│   ├── approval-form.json
│   └── trip-summary.widget.json
├── signatures/
│   ├── publisher.sig
│   ├── skill-attestor.sig
│   └── package.sig
└── attestations/
    ├── sbom.json
    ├── provenance.json
    └── safety-review.json
```

SECTION 5

Agent.package

HOCON

Defines the agent graph — the readable source layer inside the package

skill.md

Defines individual skill behavior — the readable instruction layer

tools

Define callable capabilities

policy + evals + signatures

Define what is allowed, what must work, and who to trust

agent.package

Binds all of them together as the internet-level signed artifact

HTML is to webpages as agent.package is to agents.

SECTION 6

Agent Package as a Java-Like Package

Java got several ideas right: portable executable packages, manifest, bytecode/runtime separation, sandboxing, signatures, dependency loading, versioning, and permission boundaries. An agent.package should copy that packaging discipline.

Three Ideas Combined

Java package: portable executable artifact
Browser: user-controlled sandbox and permissions
OpenClaw: agent-native runtime with LLM, tools, memory, identity, NANDA, and audit

Package Can Contain

- Declarative agent graph + signed skills
- WASM or JVM-safe executable modules
- MCP tool bindings + A2A remote agent bindings
- Memory schema + budget policy
- UI widgets + evals + attestations

The OpenClaw runtime acts like a modern, agent-native JVM/browser hybrid — interpreting packages under strict policy, not blindly running arbitrary code.

SECTION 7

Message Box vs End Points

MessageBox	End Points
Agent.Package keeps messages in a box. And the agent scans messages sequentially to act on them.	Endpoints are too risky for bidirectional execution. But in some cases (streaming, low latency, ‘read only’ operations), EndPoints play a role.

This separates **bidirectional communication** from **direct endpoint communication** - a trade off between speed and safety

SECTION 7

Local vs. Remote Execution

Runs Locally in OpenClaw	Stays Server-Side
travel@ramesh.com agent reads user preferences, calendar, prior trips, and budget constraints — if the user allows it	flights@jetblue.com exposes only a controlled service API — JetBlue's internal fraud logic, pricing algorithms, and customer database never leave the server

This separates **portable public orchestration** from **private authoritative services** — the Agentic Web version of AJAX.

SECTION 8

Reverse Engineering and Secrets

Downloadable agents can be inspected and reverse engineered. So agent creators cannot protect their IP and secrets.
The browser should assume everything inside the downloaded package is visible.

✅ Safe to Include

- Agent graph + public prompts
- Skills + tool contracts
- UI flows + memory schema
- Budget policy + evals
- Public documentation

🚫 Keep Server-Side

- API keys + private customer data
- Fraud rules + secret pricing logic
- Internal business rules
- Proprietary model weights
- Sensitive prompts + shell commands

The web works this way already: JavaScript is inspectable, but valuable private services remain on servers.

SECTION 9

Permissions and Safety

OpenClaw should treat every downloaded agent like a potentially risky web app, showing a permission screen before execution.

1

Permission Display

LLM router, token budget, data access (calendar, travel prefs), memory writes, and approved domains (jetblue.com, hilton.com)

2

Granular Controls

Allow once / session / always; deny memory or payment access; use cheaper or local model; run in private or no-memory mode

This becomes the agent equivalent of browser permissions, cookies, sandboxing, and extension controls.

SECTION 10

Token Budgets and Who Pays

Rendering HTML is nearly free. Running agents costs money. Every agent.package must declare budget expectations including estimated tokens, hard cap, cost range, max remote calls, and whether approval is required.

User Pays

User runs the agent using their own LLM router

Publisher Pays

Airline pays for customer-service agent usage

Subscription

User pays OpenClaw or LLM router monthly

1-800 Model

Company pays for inbound customer-agent calls

Escrow

Browser locks a budget before execution begins

Outcome-Based

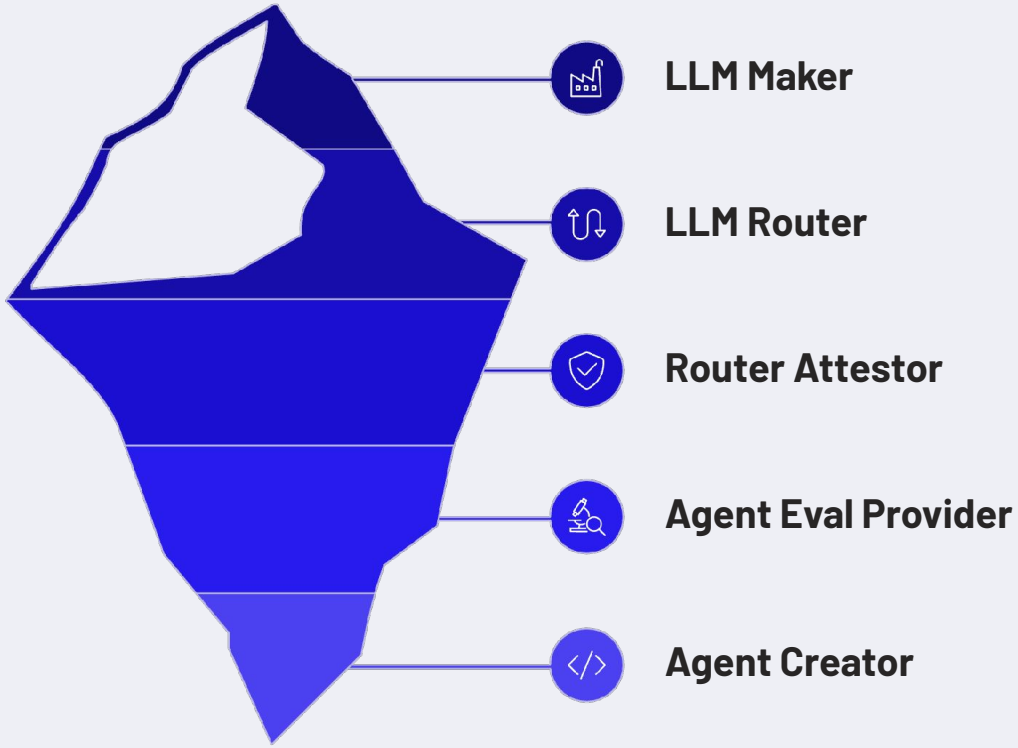
Agent receives payment after task success

The OpenClaw browser must enforce the hard cap. **No agent should be allowed to silently spend unlimited tokens.**

SECTION 11

LLM Router as First-Class Infrastructure

Agent creators should not hard-code the model. Instead, the agent package declares requirements — tool-use ability, reasoning depth, latency, privacy level, cost sensitivity — and the user's LLM router chooses the model.



This shifts power away from individual LLM providers toward **routers, evals, attestors, and agent creators**. The LLM becomes more commoditized.

The router becomes the user's intelligence broker.

SECTION 12

Tools: Client-Side and Server-Side

Client-Side Tools

Run inside the OpenClaw sandbox:

- Calendar reader + file parser
- Local memory search
- PDF summarizer + local calculator
- Personal preference lookup
- Local database query

Server-Side Tools

Run remotely:

- Airline inventory + hotel pricing
- Bank account verification
- Health record access
- Enterprise CRM + refund authorization
- Government service APIs

The package declares where each tool runs — execution location, endpoint, and required permission.

This is the agent-native version of **client-side JavaScript plus server-side APIs**.

SECTION 13

Memory as Agent Cookies + Safety

Agent memory should not repeat the mistakes of cookies and become hidden tracking infrastructure. OpenClaw should support **memory capsules**: scoped to agent identity, visible and editable by the user, revocable, portable, encrypted, permissioned, and auditable.

Ephemeral

Cleared after each session

Session

Persists within a single session

Agent-Scoped

Tied to a specific agent identity

Publisher-Scoped

Shared across a publisher's agents

User-Global

Available across all agents with permission

Enterprise-Governed

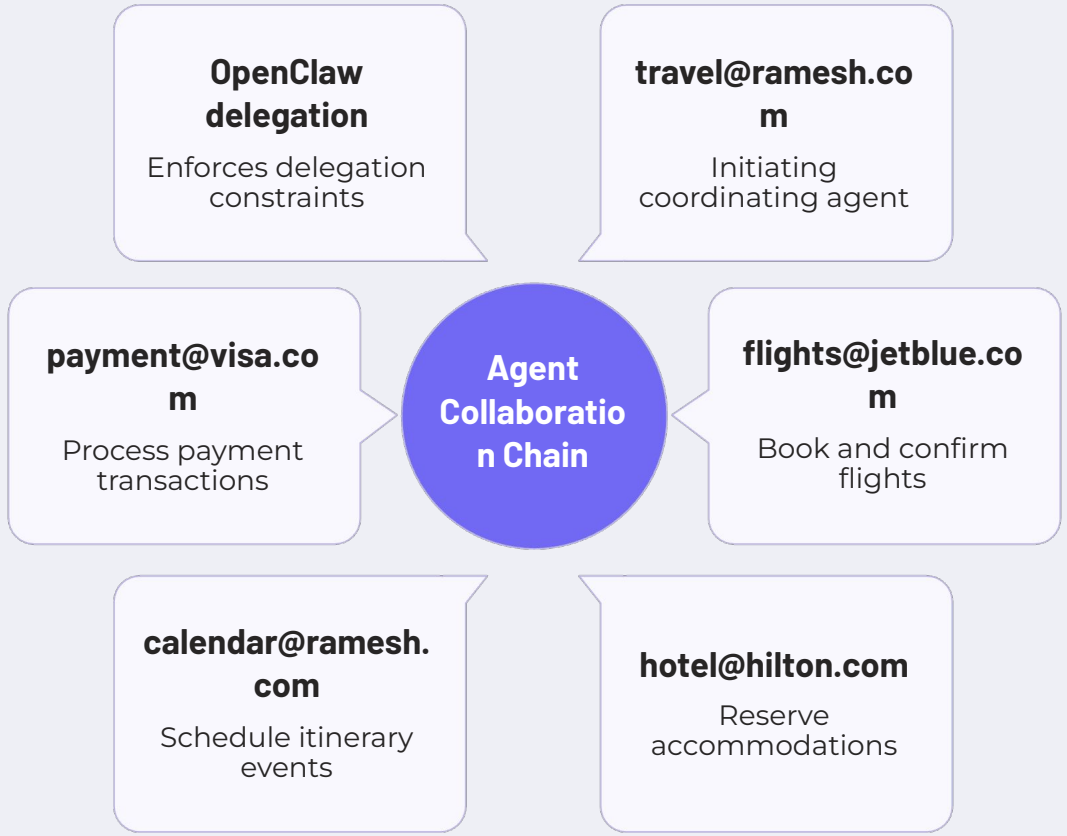
Managed under organizational policy

SECTION 14

Agent Collaboration

One downloaded agent should be able to call other agents — but collaboration must be constrained.

OpenClaw enforces **max delegation depth, approved sub-agent lists, budget per sub-agent, memory sharing policy, and a full audit trail.**



An agent should not be allowed to recursively spawn an uncontrolled network of agents. This turns agent collaboration into a **browser-governed capability** rather than a hidden server-side cascade.

SECTION 15

NANDA Index as the URL/DNS Layer

The Agentic Web needs NANDA resolution. Users and agents refer to addresses like **refunds@jetblue.com** or **travel@ramesh.com**. OpenClaw resolves these through a 7-step process.

Parse + Query

Parse the agent locator and query the NANDA Index

Retrieve + Verify

Retrieve signed AgentFacts, verify publisher and package location

Fetch + Verify Package

Fetch agent.package and verify its signature

Load Under Policy

Load the package into OpenClaw runtime under enforced policy

NANDA resolves agents. OpenClaw runs agents. agent.package transports agents. Signed skills credential capabilities.

SECTION 16

Browser vs. Server vs. NANDA Index Responsibilities

OpenClaw Browser

Package download + verification, permission UI, LLM router binding, local tool execution, memory vault, budget enforcement, audit log, plugin system

Agent Origin Server

Serves agent.package + AgentFacts, remote tools + private APIs, eval results, attestations, package updates, billing, and publisher-side policy

NANDA Index

Agent discovery + identity, package location, publisher verification, registry federation, trust metadata, version pointers, capability metadata

SECTION 17

Plugin Model

OpenClaw should support plugins just as browsers support extensions — becoming the neutral runtime into which competing agentic infrastructure providers can plug in.



LLM Router Plugin

e.g., OpenRouter — routes LLM calls across providers



Memory Provider Plugin

e.g., Mem0-like — provides persistent, scoped memory



Payment Plugin

e.g., Visa — payment authorization and budget enforcement



NANDA Plugin

Agent resolution and discovery layer



MCP / A2A Plugin

Connects tools and calls remote agents



Neuro-SAN Plugin

Runs HOCON agent graphs declaratively

SECTION 18

Attestation and Credentialing

A downloaded agent package creates a software supply-chain problem. Skills, tools, evals, and remote agents may come from many different authors. The browser needs credentialing at every layer.

Package	Skill	Tool	Eval	Router
This package came from this publisher	This skill was reviewed or certified	This tool does what it claims	This eval is meaningful and not fake	This router passes the agent's evals

This creates a new ecosystem of **attestation agencies, skill credentialers, router evaluators, and safety reviewers**.

SECTION 19

Search Engine for Agents → Returns Agent Programs

The Agentic Web needs search — but agent search is harder than web search because agents may not have public hyperlinks. The best agent search engine combines taxonomy, semantic search, capability graph, usage graph, eval graph, and trust graph.

> Here are five refund agents: **1.** Official JetBlue refund agent **2.** Fastest agent **3.** Cheapest agent **4.** Highest-rated agent **5.** Best enterprise-compliant agent — with attestation status, eval pass rates, and required permissions shown for each.

An agent search result should not just return links. It should return **executable, trusted choices**. This is a major new infrastructure opportunity.

SECTION 20

Reputation as a Core Primitive

Agent reputation is more than stars. OpenClaw should display publisher identity, signature status, skill credential status, eval pass rate, runtime compatibility, known vulnerabilities, cost reliability, and task success rate.

Budget Compliance

Does the agent stay within declared budget?

Approval Before Action

Does it ask before irreversible actions?

Memory Protection

Does it protect user memory and avoid prompt injection?

This creates room for **AgentRank, router benchmarks, package scanners, and agent safety labs**.

SECTION 21

HOCON, Neuro-SAN, and OpenClaw

Neuro-SAN demonstrates that multi-agent systems can be represented declaratively through HOCON-like configuration — an important proof point. OpenClaw builds on that idea and generalizes it into a browser-native open web architecture.

- **Neuro-SAN / HOCON**

  Declares agent networks

- **agent.package**

  Portable signed bundle for distribution

- **NANDA**

  Resolves agent identities and package locations

- **MCP / A2A**

  Tool and agent communication protocols

- **OpenClaw**

  Neutral runtime layer — any framework can publish downloadable agents

SECTION 22

The New Open Agentic Web Stack



This gives us an **internet-native architecture** rather than another closed agent platform.

SECTION 23

Why This Avoids Walled Gardens

❌ Closed Systems Today

- One vendor, one app, one model
- One tool ecosystem
- One memory store
- One agent marketplace

Useful short-term. **Dangerous long-term.**

✅ The Open Alternative

- Anyone can publish an agent
- Any browser can run it
- Any router can supply intelligence
- Any memory provider can be used
- Any attestor can certify it
- NANDA can resolve it; OpenClaw can execute it

> Agents should be publishable like websites, downloadable like apps, sandboxed like browser code, composable like APIs, and governed like critical infrastructure.

SECTION 24

Hackathon: Build the First Mini Internet of Downloadable AI Agents

A Cognizant + MIT / Project NANDA hackathon framed around building the **missing primitives of the Agentic Web** — not merely agents, but the infrastructure layer beneath them.

Browser & Runtime

OpenClaw agent-native browser, agent.package spec, skill.md signing system

Discovery & Search

NANDA resolver, agent finder, agent search taxonomy, capability graph

Trust & Evals

LLM router plugin, router attestation system, agent eval harness, package scanner

Memory & Payments

Memory capsule plugin, budget and payment simulator, local/remote tool router

Compatibility

HOCON-to-agent.package compiler, Neuro-SAN package adapter, OpenClaw developer tools

Demo: User types travel@ramesh.com → NANDA resolves → package downloads and verifies → permissions shown → agent runs locally → calls airline/hotel/calendar agents → token cap enforced → audit log shows every call, tool, model, and cost.

The Foundation for an Open Agentic Web

> OpenClaw becomes the browser for the open Agentic Web: it downloads signed agent.package bundles, verifies their skills, tools, policies, evals, and permissions, binds them to the user's LLM router, memory, identity, and budget, and runs them safely while NANDA provides discovery and resolution.

OpenClaw	**NANDA**	**agent.package**	**skill.md**	**LLM Routers**	**Attestors**
The browser	The resolver	The webpage	The signed instruction layer	Intelligence providers	Trust providers

The web made documents portable.
Mobile made apps installable.
Cloud made APIs callable.
The Agentic Web will make agents downloadable.

END